THISTLE: AN APPLICATION-LEVEL INTERFACE FOR LOCAL LLM INFERENCE

WORK-IN-PROGRESS MANUSCRIPT

Tolulope Oshinowo
Princeton University
toshinowo@princeton.edu

ABSTRACT

The growing availability of downloadable language models offers developers an alternative to externally hosted inference, but integrating local models into mobile applications remains challenging. Developers must coordinate model assets, native execution, conversations, generation settings, streaming, and resource lifetimes. We present Thistle, an open-source React Native harness that organizes these responsibilities behind a typed application-level interface built over Nitro Modules and `llama.cpp`. Its contract centralizes model defaults, reserves output capacity when enforcing rendered-input limits, and separates answer delivery from completion metadata. We evaluate six GGUF checkpoints through host generation tests, JavaScript interface tests, and CPU trials on one M2 Mac, alongside matched Thistle and `llama.rn` chat implementations. These findings demonstrate an implemented approach to local inference integration while highlighting the distinction between inference success and application reliability.

Keywords On-device inference · React Native · Developer tooling · Local-first computing · AI infrastructure

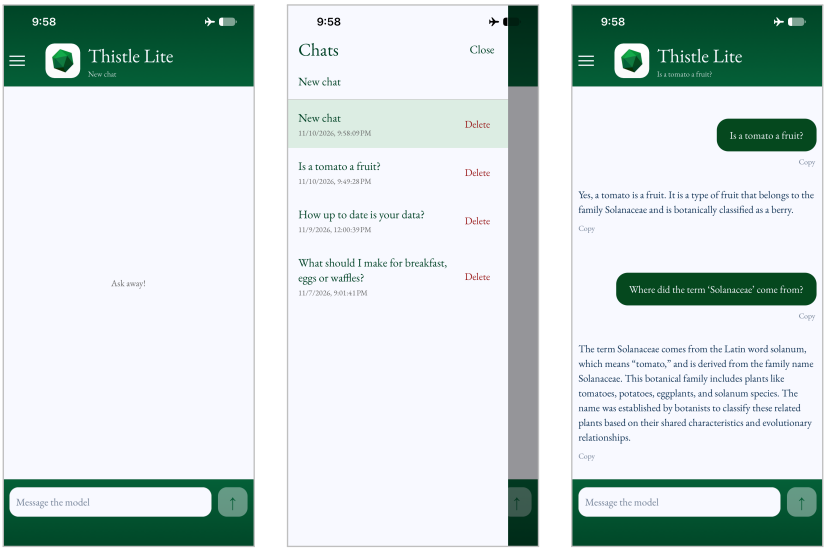


Figure 1: Thistle Lite, a proof-of-concept demonstration application built using Thistle, illustrating a lightweight conversational interface for interacting with a locally running language model. The interface follows familiar design conventions found in general-purpose conversational AI applications, with three views shown: a new-chat screen (left), a conversation selection screen (center), and an active conversation in which the user explores the etymology and botanical classification of the tomato plant family (right). The application demonstrates how Thistle can be integrated into a familiar, consumer-facing mobile experience. The demo is available at `https://thistle.earth/demo`.

1 Introduction

While the public release of ChatGPT in late 2022 brought conversational large language models (LLMs) to a broader audience, it was the GPT-3.5 Turbo API in the following year that truly catalyzed the expansion of specialized LLM-based applications for developers [1, 2]. In parallel, researchers began evaluating these models and contributing their findings to the growing broader ecosystem [3–5]. Together, these developments contributed to the proliferation of the term "AI wrapper" [6], commonly used to describe applications built around existing AI models and services. Typically, "AI wrappers" integrate an externally operated LLM through an API-mediated request-response cycle where user inputs are transmitted to the model, processed remotely, and returned to the application, and the resulting outputs are presented or incorporated into an application-specific functionality. This approach offers both opportunities and limitations. While access to existing models lowers entry barriers and enables application innovation without requiring developers to train foundation models [7], dependence on external providers can constrain customization, differentiation, and control over complementary assets [7, 8]. It also introduces dependencies on providers' pricing, service availability, and model behavior, alongside questions of data governance and processing [9, 10]. More broadly, cloud research highlights portability and interoperability limitations as potential sources of vendor lock-in [11].

These limitations have increasingly motivated interest in alternative approaches that reduce reliance on externally hosted models. In particular, advances in on-device models, compression, acceleration, and resource management have made it increasingly feasible to run open weight downloadable models directly on personal devices, enabling offline inference, greater control over model selection, and reduced dependence on external providers [12–14]. This approach also aligns with local-first computing's emphasis on data ownership and continued operation independent of cloud services [15]. However, unlike API-based services, local inference requires developers to manage much of the underlying infrastructure themselves. This is particularly challenging in mobile applications, where model integration involves coordinating model assets, native bindings, conversation templates, inference parameters, streaming, and resource management. Although tools such as `llama.cpp` provide underlying inference capabilities, integrating these components into a cohesive and accessible application-level framework remains a significant development challenge [16–18]. Consequently, while local models offer greater control over inference, translating that capability into a usable mobile application remains considerably less straightforward.

To address this integration gap, we introduce Thistle, an open-source application-level harness that provides a unified React Native interface for local LLM inference. Built over Nitro Modules and a shared `llama.cpp` engine, Thistle coordinates model packaging, structured conversations, streaming generation, request budgets, and explicit resource unloading. Its contribution is not a new inference algorithm or native binding, but a coordinated application-level interface that abstracts the complexity of integrating local models into mobile applications [16–18]. The interface also provides model-specific generation defaults, input limits that reserve capacity for output tokens, and separate delivery of generated text and completion metadata. We evaluate the implementation through host-side generation tests, mocked JavaScript tests, and matched iOS Simulator builds against `llama.rn`. Altogether, these results establish Thistle as an implemented and testable artifact while highlighting the remaining challenges in reliable local inference integration.

2 Related Work

Making downloadable language models available does not necessarily make them practical to deploy. Work on model openness has characterized access as a gradient of release arrangements, distinguishing hosted services, API access, and the release of model weights [19]. However, access to weights addresses only one part of the deployment process. Foundation models remain embedded in broader technical structures, and their capabilities and limitations carry through to the applications built around them [20]. For application developers, the relevant distinction is therefore between *model availability* and *practical access*: the ability to package, configure, execute, and manage a model within an application. This shifts attention from what developers can obtain to the infrastructure required to put those resources to use.

Research on on-device inference has addressed many of the computational challenges involved in deploying models outside centralized infrastructure. Techniques including quantization, compression, hardware acceleration, and memory management have made local execution increasingly feasible under device resource constraints [14, 21, 22]. Alongside these advances, tools such as `llama.cpp` provide portable inference runtimes for loading model weights, tokenizing inputs, and generating outputs across hardware environments [16]. Cross-language frameworks address a complementary integration problem: Nitro Modules, for example, connects JavaScript and TypeScript application code to native implementations written in languages such as C++, Swift, and Kotlin [18]. These components provide important foundations for local inference, but address different layers of the deployment stack. An inference runtime executes the model, while a native bridge connects that execution to application code. Neither, on its own, establishes how

an application should organize model configuration, conversation handling, generation limits, streamed responses, or resource lifetimes.

Existing React Native libraries bring these underlying capabilities closer to application developers. In particular, `llama.rn` provides an interface for local inference that includes structured conversations, model-specific chat templates, streaming, context management, and cancellation, alongside capabilities such as tool calling and multimodal processing [17]. Thistle builds on this ecosystem rather than introducing another inference runtime or native binding. Its contribution lies in how application-level responsibilities are organized behind a shared interface: model-specific generation defaults, input budgets that reserve capacity for output, separate delivery of answer text and completion metadata, and explicit resource unloading. This represents a particular approach to the application-facing contract for local inference. Rather than treating access to an inference engine as sufficient, Thistle makes the policies governing how applications configure and consume that engine explicit.

3 Architecture and Implementation

Integrating local inference into a mobile application requires coordinating responsibilities across the application and native execution layers. Developers must manage model loading, conversation formatting, generation parameters, context limits, asynchronous responses, and resource lifetimes. Thistle organizes these responsibilities behind a shared TypeScript interface, with platform-specific bindings delegating inference operations to a common C++ engine. This section describes the architecture from the application-facing contract through native execution and resource management, followed by the implementation of model loading, conversation budgeting, and streamed generation. Figure 2 illustrates how these components connect. Further implementation details, source code, and usage documentation are available in the Thistle GitHub repository.

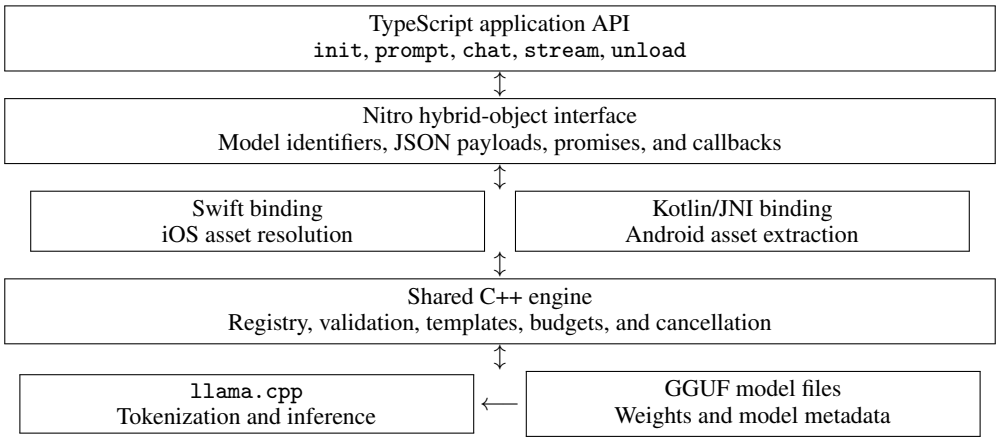


Figure 2: Logical architecture of Thistle. Bidirectional arrows indicate requests and returned results or streamed text; the model-file arrow indicates loading. Asset resolution supplies a filesystem path to the shared engine.

3.1 A typed application contract over a narrow bridge

Thistle exposes five operations for working with a local model. `Thistle.init` loads a compatible model from a filesystem path or application asset reference, performs a warmup, and returns a model instance with associated generation defaults. The instance exposes three ways to submit requests: `prompt` accepts a user request and optional preceding context, `chat` accepts a structured sequence of system, user, and assistant messages, and `stream` accepts the same conversation structure while delivering generated answer fragments through a callback. Both `chat` and `stream` return completion information alongside the generated answer, including token counts, separated reasoning when supported, and whether generation stopped naturally or reached its output limit. Finally, `unload` removes the model from use and signals cancellation to requests still executing against it. These operations provide a consistent interface for loading models, submitting conversations, consuming generated responses, and releasing resources. The application remains responsible for maintaining conversation history, rendering responses, and deciding when to unload a model.

The TypeScript interface connects to native implementations through Nitro Modules, which generates connecting code from shared interface definitions. This allows application code to invoke operations implemented in Swift, Kotlin, and C++ without maintaining separate handwritten bridges for each platform. Thistle passes structured requests and completions across this boundary as JSON strings. Native methods consequently receive a consistent representation of

conversation messages and generation options, which the shared engine parses and validates. This approach reduces the number of platform-specific representations that must be maintained, but introduces serialization and runtime parsing overhead. Generated method signatures also guarantee only that values conform to the declared interface: a valid string may still contain an invalid request. TypeScript checks and generated bindings therefore complement rather than replace validation within the native engine.

3.2 Shared semantics and platform-specific responsibilities

The shared C++ engine defines the semantics of an inference request across platforms. It maintains the model registry, formats conversations using model-specific templates, validates request parameters, enforces token budgets, configures sampling, and assembles completion results. Keeping these responsibilities in one implementation avoids independently reproducing request handling and generation behavior in the iOS and Android layers. The platform bindings instead handle operating-system-specific operations, including resolving model assets, converting strings, invoking the shared engine, and delivering results and errors to application code. For example, iOS can resolve a model from the application bundle, whereas Android may need to extract a packaged asset to a filesystem path before loading it. Although both platforms expose the same application operations, their underlying packaging and execution paths remain distinct. Shared engine semantics therefore provide a common implementation of request handling, not a guarantee of identical behavior across devices or operating systems.

Model ownership and generation requests have separate lifetimes. A registry associates model identifiers with shared native ownership, allowing loaded weights to remain available across successive requests. Each request acquires ownership of its model and a generation lock, ensuring that only one request generates at a time on a given instance. Temporary inference contexts and sampling resources are created for individual requests and released afterward; conversation history is not retained by the engine. Applications must therefore resubmit the messages needed for each turn. When `unload` is called, the engine removes the model identifier from the registry and sets an atomic cancellation flag that active requests can observe across threads. Requests retain ownership of the model until execution exits, preventing resources from being released while still in use. An inference abort callback checks this flag during generation, and the engine checks it again before returning results. Cancellation thus prevents continued execution without guaranteeing immediate reclamation of every allocation. Applications may instantiate multiple models, but each additional instance introduces further weight and execution-state costs; concurrent instances do not imply efficient parallel inference.

```
import { Thistle, type ThistleMessage } from 'react-native-thistle';

const model = await Thistle.init(modelPath, {
  maxContextSize: 2048, maxOutputTokens: 64,
  temperature: 0, seed: 42, enableThinking: false,
});
try {
  const messages: ThistleMessage[] = [
    { role: 'user', content: 'Name a river in Paris.' },
  ];
  const first = await model.chat(messages);
  messages.push(
    { role: 'assistant', content: first.text },
    { role: 'user', content: 'Describe its surroundings.' },
  );
  await model.stream(messages, delta => appendAnswer(delta));
} finally {
  model.unload();
}
```

Figure 3: A two-turn request using Thistle's public interface. Generation defaults set at initialization are reused by chat and streaming calls. The model path and answer-rendering callback are supplied by the application, which also maintains the transcript; the corresponding example is typechecked with the integration artifact.

3.3 Implementation details

Thistle supports compatible decoder-only GGUF text-generation models. During initialization, the application supplies a model path or packaged asset reference, which the platform binding resolves into a filesystem path accessible to the inference engine. On iOS, this involves resolving paths or assets within the application bundle. On Android, packaged

assets are copied into the application cache because the engine requires a filesystem path. The Android extraction process writes to a temporary file before renaming it, and incorporates the application's update time into cached filenames to avoid reusing outdated copies after an application update. These platform-specific loading paths introduce different storage and first-load costs. A GGUF file's format alone does not guarantee architectural compatibility or that its weights and execution state will fit within available memory; unsupported model types are rejected by the engine. Initialization also prepares conversation formatting from supported model metadata or an explicitly supplied template override. The public `unload` operation is idempotent, allowing applications to release a model without risking duplicate destruction.

Each inference request contains a sequence of messages with `system`, `user`, or `assistant` roles. The engine renders these messages using the selected model's chat template before tokenizing the resulting conversation. Consequently, context budgeting accounts for template and control tokens in addition to the text supplied by the application. The effective context allocation is bounded by both the requested context size and the model's training-context limit, while the output budget must remain smaller than the allocation. Thistle calculates the effective input budget as the smaller of the requested input limit and the available context after reserving the requested output capacity. If the rendered conversation exceeds this budget, the request fails with `context_length_exceeded` rather than silently removing earlier messages. Additional errors distinguish invalid templates, unsupported models, context-creation failures, and lifecycle violations. Applications may also supply instruction scaffolding, including the built-in `xylem` preset, or serialize structured data into textual context. These facilities influence the input presented to a model; they do not introduce trained capabilities, retrieval, or independent multimodal processing. Finally, token limits constrain the rendered request rather than total memory consumption, which also depends on model weights, inference state, and backend buffers.

Model loading and generation execute asynchronously through native promises. During streaming, the engine sends generated answer-text fragments through a callback and returns completion metadata when the request finishes. The native bindings convert strings and byte arrays across their respective language boundaries, including the JNI boundary on Android, using standard UTF-8 handling rather than treating arbitrary text as JNI modified UTF-8. The TypeScript wrapper stops forwarding callbacks when a request becomes inactive or its model is unloaded. Errors thrown by an application's streaming callback are captured and propagated by rejecting the request after native completion, rather than being thrown directly on the callback dispatcher. Callback errors do not independently cancel native generation; cancellation is handled through the model's unloading mechanism. For models that support reasoning delimiters, the engine can separate supported `<think>` blocks from the answer stream and return them as reasoning metadata. A configurable reasoning-token budget may cause the engine to insert closing tokens when sufficient output capacity remains. This is delimiter-based handling rather than a general mechanism for detecting internal reasoning or sensitive content. Accordingly, the time to the first visible answer fragment may differ from the time at which the model first generates a token.

4 Evaluation

We evaluate Thistle at three boundaries: native inference behavior, application integration, and execution in a React Native application. First, we test six compact model checkpoints against the request and lifecycle contract described in Section 3. We then implement the same chat workflow using Thistle and `llama.rn`, comparing the responsibilities expressed by each adapter and executing both implementations in separate iOS Simulator builds. JavaScript tests and host-side timing and memory measurements provide additional evidence about the interface and its execution. These experiments address distinct questions: whether the native engine satisfies its contract, how applications interact with each library, and whether the integrated application executes successfully. We do not treat success at one boundary as evidence of success at another.

4.1 Model selection and study scope

We selected six compact GGUF checkpoints to examine compatibility across architectures, conversation templates, and storage requirements. Model selection combined ecosystem adoption rankings with a deployment class centered on approximately two billion parameters. A November 2026 Hugging Face download ranking informed the initial candidate pool, while model cards identified checkpoints comparable in scale to Qwen3.5-2B [23, 24]. The final set comprised Qwen3.5-2B, Gemma 4 E2B, SmolVLM2-2.2B, Cosmos-Reason2-2B, MiniCPM5-2B, and Granite-3.3-2B-Instruct [25–29]. DeepSeek and Llama were excluded because the checkpoints considered exceeded the target parameter class, not because of their adoption standing. All six post-trained GGUF files were included in the example application's assets.

Table 1 reports storage size and GGUF metadata. Model names do not necessarily correspond to stored tensor-element counts: Gemma E2B contains approximately 4.65 billion stored elements, while MiniCPM5 and Granite contain approximately 2.52 and 2.53 billion, respectively. Stored elements should not be interpreted as full multimodal or effective active-parameter counts. All files declare Q4_K_M quantization, but the set is not a size-matched performance comparison. Cosmos uses a `qwen3vl` backend, whereas Qwen3.5 is the selected Qwen checkpoint. All requests used the same text-generation workflow; we did not evaluate vision, audio, or projector functionality.

Table 1: Selected GGUF assets. Storage is in GiB; stored tensor elements are in billions. Architecture and quantization labels come from GGUF metadata.

Checkpoint	Architecture	GiB	Elements	Quantization
Qwen3.5-2B	qwen35	1.193	1.882	Q4_K_M
Gemma 4 E2B-it	gemma4	2.893	4.647	Q4_K_M
SmolVLM2-2.2B-Instruct	llama	1.036	1.813	Q4_K_M
Cosmos-Reason2-2B	qwen3vl	1.194	2.032	Q4_K_M
MiniCPM5-2B	llama	1.454	2.517	Q4_K_M
Granite-3.3-2B-Instruct	granite	1.439	2.534	Q4_K_M

4.2 Environment and procedure

All six checkpoints were evaluated on an Apple M2 Mac with 8 GiB of unified memory and macOS 26.5.1. We used unchanged engine sources and SHA-256-verified executables, compiled with Apple Clang 21.0.0 and CMake's `RelWithDebInfo` configuration (`-O2 -g -DNDEBUG`). Execution used the CPU rather than Metal or GPU acceleration, with Apple Accelerate/BLAS and four inference and batch threads enabled. Correctness binaries enabled Address-Sanitizer; timing binaries did not. These measurements are native macOS CPU runs, not simulator or physical-device measurements.

The checkout was Thistle 0.6.0 at revision `359b43c6cb04`. The vendored `llama.cpp` identifier, `4dc37c4b28e9`, denotes a Git tree rather than an upstream commit. The `evaluation/` archive records source and executable fingerprints, model and template hashes, metadata, requests, completions, and logs. Existing runtime assertions and failure conditions were left unchanged. The timing harness gained an optional trial-count argument but retained its original request and measurement logic; its additional binary used matching CPU build settings. The original evidence remains intact, with supplementary trials recorded separately. This allowed us to summarize five generation observations per checkpoint without treating repeated runs on one machine as independent-system replications.

Models ran sequentially in filename order. For each checkpoint, separate processes executed the existing `model_smoke` and `engine_test` suites, followed by `engine_bench`. Each invocation had a five-minute timeout, and a failure did not prevent subsequent models from running. The smoke suite tested warmup, question answering, stopping, history-dependent follow-up and recall, UTF-8 input, control-token text, output limits, and unloading. The contract suite additionally tested streaming reconstruction, validation errors, context overflow, reasoning budgets, explicit template overrides, and unloading during generation.

Not all assertions test the same property. Some concern runtime behavior, while others depend on a model producing a particular answer or reasoning length. For example, recalling a supplied name is different from rejecting an invalid request, and producing an expected reasoning length is different from reconstructing streamed text. Assertions involving explicit template overrides also exercise a configuration distinct from the model's embedded conversation template. We report these outcomes separately rather than combining them into a single compatibility score. Invocation logs record failed assertions and process termination, so a suite that did not complete is not counted as successful.

For timing, each process loaded one model and performed the engine's one-token warmup before streaming requests with greedy decoding, seed 42, a 2,048-token context, and a 64-token output budget. Thinking was disabled, with no separate reasoning cap. The fixed prompt was: "Write a detailed description of a river flowing through a forest. Continue for at least one hundred words." We measured five generation trials per model: three initial trials and two additional trials under a second load on the same Mac. First-answer-delta latency measures the interval from the start of streaming to the first callback. Request duration measures the interval until the call returns. Throughput divides completion tokens by full request duration, including prefill and callbacks, rather than measuring isolated decoding.

Timing medians use all five trials. Load and post-unload RSS medians use two completed processes. Peak RSS is the process high-water mark reported by `getrusage`; post-unload RSS comes from macOS `task_info`. These measurements describe the tested host processes, not mobile memory requirements or complete resource reclamation. Thermal state and host activity were uncontrolled, limiting comparisons across models.

4.3 Functional results and failure analysis

All six checkpoints loaded and generated. Across the exercised runtime and request-contract checks, the models handled context overflow, invalid inputs, UTF-8 streaming with embedded templates, answer reconstruction, output limits, and unload cancellation. Granite completed its cancellation checks on a diagnostic retry after its primary contract run timed out. Table 2 separates these runtime checks from assertions tied to particular models or templates. Assertion totals vary with streamed chunk counts and should not be interpreted as comparable reliability scores.

No AddressSanitizer diagnostics were emitted during the exercised paths. Separately, `yarn test -runInBand` passed all 57 tests across six JavaScript suites, and `yarn typecheck` completed successfully. The JavaScript suites include mocked Nitro/native calls and an example conversation integration test; they do not establish end-to-end behavior of the Swift or Kotlin/JNI implementations. The results therefore distinguish interface validation from native execution: JavaScript tests verify behavior under their supplied mocks, while the native suites exercise model loading and generation through the shared engine.

Table 2: Behavioral outcomes from the existing native suites. Q: Qwen3.5; G: Gemma 4 E2B; S: SmolVLM2; C: Cosmos; M: MiniCPM5; I: IBM Granite. Pass indicates that the exercised assertions passed, not general certification. Streaming rows above the divider use embedded templates.

Behavior	Q	G	S	C	M	I
Runtime and request-contract checks						
Load and generate	Pass	Pass	Pass	Pass	Pass	Pass
Reject context overflow	Pass	Pass	Pass	Pass	Pass	Pass
Reject invalid inputs	Pass	Pass	Pass	Pass	Pass	Pass
Valid UTF-8 stream chunks	Pass	Pass	Pass	Pass	Pass	Pass
Exact answer reconstruction	Pass	Pass	Pass	Pass	Pass	Pass
Enforce output limits	Pass	Pass	Pass	Pass	Pass	Pass
Cancel generation on unload	Pass	Pass	Pass	Pass	Pass	Pass
Model- and template-specific assertions						
Hide template-opened reasoning tags	Pass	Pass	Pass	Pass	Fail	Pass
Forced reasoning-template expectation	Fail	Pass	Pass	Pass	Fail	Pass
Nonempty built-in ChatML answer	Fail	Pass	Pass	Pass	Fail	Pass
Recall an earlier supplied name	Pass	Pass	Fail	Pass	Pass	Pass

Qwen3.5 failed two assertions involving explicit template overrides. With a template that opened a `<think>` block, the suite expected exactly six reasoning tokens and nonempty reasoning; the completion instead reported two reasoning tokens and an empty reasoning string. This was not a demonstrated budget overrun: the embedded-template check with an eight-token cap passed. A separate assertion expecting a nonempty answer under the built-in ChatML override also failed. The completion was not printed, so the cause remains undetermined.

MiniCPM5 passed all 11 smoke assertions and 205 of 208 contract assertions. Its three failures concerned hiding template-opened reasoning and its closing tag, separating the answer under a forced-reasoning template, and producing a nonempty answer under the built-in ChatML override. The recorded forced-template completion contained the answer text `</think>` after six reasoning tokens. These failures distinguish ordinary generation with an embedded template from answer separation under explicit overrides. They also illustrate why reasoning-token counts alone are insufficient to validate the application-facing answer: a completion can remain within its requested reasoning budget while returning an empty answer or exposing a delimiter the interface is intended to suppress.

Granite passed all 11 smoke assertions. Its primary contract run recorded 323 passing assertions before timing out after the ChatML-override unload check; its benchmark also timed out without producing measurements. A diagnostic retry used the same binaries, requests, and timeout. The benchmark completed in 33.9 seconds, and all 332 contract assertions passed in 124.6 seconds. The incomplete primary runs are not counted as passes, and the retry does not explain the earlier timeouts.

SmolVLM2 failed the name-recall assertion: it answered that no name had been provided despite the conversation history containing "Zorblat." Its history-dependent river follow-up and full contract suite passed, so this result does not establish that history was omitted from other requests. Similarly, handling control-token text without leaking selected markers does not establish prompt-injection resistance. Across the 18 primary native invocations, Qwen3.5's and MiniCPM5's contract suites and SmolVLM2's smoke suite exited nonzero because of assertions; two Granite invocations timed out, and 13 completed successfully. The separately retained Granite retries both completed successfully. Reporting assertion failures, timeouts, and completed runs separately avoids collapsing distinct outcomes into a single model score.

4.4 Matched application-integration artifact

To examine how the libraries are used within an application, we implemented the same minimal chat workflow with Thistle 0.6.0 and `llama.rn` 0.12.9 [17]. Both implementations shared React Native 0.85.0, React 19.2.3, the interface, transcript state, and interaction protocol. Separate adapters handled model loading, completion, streaming, and release. We built each provider separately for ARM64 iOS Simulator using Xcode 26.6 and the iOS 26.5 SDK. An attempted combined build encountered conflicting vendored headers and is not counted as either standalone result. The `llama.rn` build used its packaged source rather than downloaded native artifacts.

Both adapters loaded the same SmolVLM2 checkpoint from a provisioned filesystem path, using a 2,048-token context, greedy decoding, seed 42, and a 64-token output limit. The task asked the model to describe a river flowing through Paris, resubmitted the answer as conversation history, and then requested a streamed description of a forest beside that river. Checks required nonempty answers, exact reconstruction from streamed deltas, and output-limit compliance. Both implementations released resources in `finally`. Because the task supplied a filesystem path, model acquisition and native asset packaging were outside the comparison. The `evaluation/integration-study/` archive contains the sources, package and provider-specific CocoaPods locks, tests, and native observations.

Table 3 summarizes responsibilities identified from the implemented adapters and inspected source. It is not a ranking of feature presence or absence. Both libraries support structured messages, streaming, and release. Thistle concentrates model defaults and input/output reservation in its model-facing contract, whereas `llama.rn` exposes a broader context and completion interface. The adapters contain 23 and 30 nonblank lines, respectively, excluding shared code. This formatting-dependent difference does not measure developer time, usability, or maintenance effort.

Typechecking against the installed declarations and Metro bundling succeeded. All 12 adapter tests passed with mocked interfaces, covering configuration mapping, transcript forwarding, stream reconstruction, error propagation, output checks, and cleanup. These tests are separate from the 57 existing JavaScript tests and do not establish inference correctness. The comparison also has execution-level differences: the task uses a non-reasoning text model, so `llama.rn`'s raw token stream is not a general substitute for Thistle's answer-only stream on reasoning models. Engine versions, sampling defaults, context reuse, and backend policies differ. Thistle selects its native backend automatically, while the `llama.rn` adapter requests zero GPU layers. We therefore use the comparison to characterize application responsibilities, not to compare inference performance or answer quality under equivalent backends.

Table 3: Responsibilities in the inspected APIs. Optional packaging support is source-inspected, not exercised by the path-based task.

Responsibility	Thistle 0.6.0	llama.rn 0.12.9
Model provisioning	App supplies weights; optional asset-selection setup CLI	App supplies model path or asset
Native dependency	Nitro peer and generated bindings	New-Architecture JSI bindings; no Nitro peer
Request settings	Model defaults with per-call overrides	Options on `completion`; context allocation at initialization
Chat and streaming	Role messages; separate answer-delta operation	Role messages; optional token callback and parsed content fields
Budget policy	Rendered-input cap reserves requested output; rejects excess	Context/output controls and configurable context shifting; different overflow contract
Stop and release	`unload` removes model and aborts in-flight work	Separate `stopCompletion` and asynchronous `release`
Transcript and UI	Application responsibility	Application responsibility

Both standalone Release builds succeeded and ran on the iPhone 17 Pro simulator with iOS 26.5, on the same M2 host. `llama.rn` produced nonempty first and follow-up responses, reporting 63 completion tokens for each. Its 64 streamed chunks reconstructed the final follow-up exactly; callback counts and reported token counts are distinct observations. Thistle failed the nonempty-stream check in two attempts. In the diagnostic repeat, its first answer was "ipzig" with one reported completion token, followed by an empty answer with zero reported tokens and no callbacks. Both the original attempt and diagnostic repeat are retained. Only observation capture changed between them; the prompts and provider adapter remained the same.

These results establish neither semantic answer quality nor the cause of the divergent behavior. Differences in backend policy and engine versions prevent attributing the outcome solely to API design. They do show that successful host-engine checks and mocked adapter tests do not guarantee successful native application behavior. The Thistle simulator failure is therefore a concrete limitation of the evaluated artifact under this protocol and native configuration.

4.5 Supporting runtime observations

All 30 measured streaming requests produced 64 completion tokens, a length finish reason, no recorded reasoning tokens, and nonempty answers whose deltas reconstructed the final text exactly. Granite's first three trials came from its successful diagnostic retry; the remaining models contribute two additional trials each. Table 4 reports median first-answer-delta latency and whole-request throughput. Measurements exclude AddressSanitizer overhead but include formatting, tokenization, context creation, prefill, generation, and callbacks.

Throughput ranged from 20.20–30.28 tokens/s for Qwen, 19.91–27.27 for Gemma, 27.09–44.85 for SmolVLM2, 32.79–43.62 for Cosmos, 6.61–28.96 for MiniCPM5, and 14.88–22.06 for Granite. MiniCPM5's first-delta latency ranged from 323.2 to 1237.9 ms, illustrating variability within this small workload. Median immediate post-unload RSS ranged from 16.9 to 67.5 MiB. These values do not establish complete resource reclamation or characterize long-running behavior.

The measurements are descriptive rather than comparative: host activity was uncontrolled, Granite's initial timeouts interrupted its primary measurements, and five trials on one machine are insufficient for a robust model ranking. Exact stream reconstruction establishes agreement between the chunks and final answer, not factual accuracy, usefulness, or mobile responsiveness. These observations supplement the contract and application tests but do not replace them.

Table 4: Supporting host CPU observations, not a model-family ranking. First-answer-delta latency and whole-request throughput are medians of five 64-token trials. Load and post-unload RSS are medians across two completed processes; peak RSS is their maximum. RSS is in MiB, not device memory requirements. Granite's asterisk denotes inclusion of its successful retry.

Checkpoint	Load (s)	Delta (ms)	Tokens/s	Peak RSS	After unload
Qwen3.5-2B	3.98	299.1	27.14	2484	56.8
Gemma 4 E2B-it	4.75	357.0	25.64	2730	67.5
SmolVLM2-2.2B	0.63	190.9	43.54	2391	23.3
Cosmos-Reason2-2B	4.19	199.9	42.64	2250	33.5
MiniCPM5-2B	11.81	519.6	16.47	1864	24.3
Granite-3.3-2B*	13.35	1003.5	17.61	2117	16.9

5 Discussion

Thistle's evaluation points to a distinction between making model weights available and making them usable within an application. Work on model openness characterizes access as a gradient of release arrangements, rather than a binary property [19]. Similarly, foundation models are situated within broader technical structures that shape the capabilities and limitations inherited by downstream applications [20]. Thistle addresses one part of this intervening infrastructure: the application-level operations that connect model weights to conversations, generation, streamed responses, and resource management. The six-model evaluation shows that these responsibilities can be organized behind a shared contract, but the template-override and recall failures demonstrate that the contract cannot eliminate model-specific behavior. Practical access therefore depends not only on whether weights can be obtained, but also on how models, runtimes, application interfaces, and deployment environments work together. Thistle makes this integration layer explicit and testable; it does not resolve the broader questions surrounding model openness.

This application layer complements, rather than replaces, work on efficient on-device inference. Research on compression, acceleration, and resource management addresses the computational constraints of local execution [14, 21, 22]. Thistle assumes an execution substrate and focuses on how applications use it. The relationship between `llama.cpp`, Nitro Modules, and existing React Native bindings illustrates the distinct roles of inference execution, cross-language integration, and application-facing interfaces [16–18]. The paired implementations show that libraries already expose many of the necessary primitives, including structured conversations, streaming, and resource release. Thistle's particular approach is to organize model defaults, input and output budgets, answer delivery, and lifecycle management into a single contract. However, the simulator experiment also exposes the limits of this abstraction: both applications built, but only the `llama.rn` implementation completed the scripted workflow. Similarly, Granite's successful diagnostic retry did not explain its initial timeouts. These results suggest that application contracts deserve independent validation. A working inference engine, a successful host test, and a successful application build each establish different things; none guarantees that the integrated application will behave as intended.

Local inference also raises questions about control, but does not answer them by itself. Local-first computing connects local execution to ownership and continued operation [15], while research on decentralized social media shows how

control is distributed across identity, curation, and infrastructure [30]. These perspectives suggest examining which responsibilities shift from service providers to application developers, and whether those shifts create meaningful choices for users. Research on feed curation provides a related application context: *Pantry* examines how people exercise agency over algorithmic feeds [31], while Bonsai explores intentional feed creation and the importance of transparency and configuration effort [32]. Together with work on locality [33], these studies motivate examining whether applications built with Thistle make model selection, data retention, and offline operation understandable and configurable. The present evaluation does not establish that they do. It evaluates an integration harness, not users' experiences of control. Instead, it identifies a design question for future applications: whether the flexibility afforded by local inference can be translated into choices that users can understand and exercise. Changing where inference runs may alter an application's dependencies, but does not by itself redistribute power or improve user agency.

6 Limitations and Future Work

The experiments reported here cover a specific set of models, requests, and execution environments, leaving several dimensions for further study. Here, we examined six checkpoints and text-only requests, leaving more to be explored as i t relates to compatibility with other architectures, multimodal capabilities, and broader answer quality. Host measurements were also limited to one M2 Mac and a single prompt, with five generation trials and two completed benchmark loads per model. Uncontrolled cache, thermal, and background conditions further limit performance comparisons. CPU execution provides a common baseline rather than a measure of maximum device performance, while reliance on Apple Accelerate/BLAS limits hardware independence. The application comparison also leaves important aspects of deployment unexamined: its single chat workflow used a provisioned filesystem path, excluding model acquisition and native asset packaging, while adapter-line counts provide only a limited proxy for developer effort.

These limitations suggest several directions for further systems evaluation. First, model distribution warrants investigation, including resumable downloads, integrity checks, version management, and recovery from interrupted updates. Expo Go cannot load arbitrary custom native modules, so Thistle currently requires a compatible native development client [34]. Supporting Expo development builds and automating their configuration are practical next steps; Expo Go compatibility, however, would require a supported execution path or changes to bundled capabilities rather than a configuration plugin alone. More immediately, the unsuccessful simulator follow-up requires investigation. Future work should trace the request through generation, native dispatch, and answer delivery, then repeat the matched application protocol with aligned backends where feasible. A broader systems evaluation should also vary model architectures, templates, operating systems, and device tiers, and include a direct-`llama.cpp` baseline. Physical-device studies should measure responsiveness, memory, energy consumption, thermal behavior, backgrounding, and recovery from interrupted execution. Together, these experiments would establish whether the interface behaves consistently beyond the host and simulator configurations examined here.

Beyond technical validation, the proposed application-level benefits require evaluation with developers and users. Developer studies could compare implementation time, errors, debugging practices, and maintenance across alternative interfaces. A user study of a Thistle-built application could examine whether people understand where inference occurs and can meaningfully configure model selection, data retention, and offline operation. Such a study should relate perceived control to actual permissions, logging, and network behavior, while accounting for trust and accessibility. The costs of model storage and distribution also remain unmeasured. Accordingly, lower cost, equitable access, and environmental benefits should be treated as hypotheses for evaluation rather than assumed consequences of local execution. Combining deployment audits with participant studies would help establish when the technical flexibility of local inference translates into practical benefits.

7 Conclusion

Thistle demonstrates an approach to organizing local language model inference within React Native applications through a shared application-level interface. The evaluation shows that this approach works across multiple checkpoints and supports reconstruction of streamed responses, while also exposing model-specific behaviors and integration failures that remain difficult to abstract away. In particular, the unsuccessful simulator follow-up highlights the distinction between validating individual components and establishing reliable end-to-end behavior. More broadly, Thistle provides a basis for studying how local inference infrastructure is exposed to application developers and, ultimately, to users. Whether its interface reduces integration effort, supports meaningful user control, or broadens practical access remains to be established through developer studies, deployment evaluations, and user-facing applications.

References

[1] OpenAI. Introducing ChatGPT. Product announcement, 2022. URL https://openai.com/index/chatgpt/.

[2] OpenAI. Introducing ChatGPT and Whisper APIs. Product announcement, 2023. URL https://openai.com/index/introducing-chatgpt-and-whisper-apis/. The announcement's ChatGPT API name refers to the GPT-3.5 Turbo API.

[3] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models. arXiv preprint arXiv:2303.18223, 2023. URL https://arxiv.org/abs/2303.18223.

[4] Michael A. Cusumano. Generative AI as a new innovation platform. *Communications of the ACM*, 66(10):18–21, 2023. doi: 10.1145/3615859.

[5] Chuan Yan, Ruomai Ren, Mark Huasong Meng, Liuhuo Wan, Tian Yang Ooi, and Guangdong Bai. Exploring ChatGPT app ecosystem: Distribution, deployment and security. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 1370–1382, 2024. doi: 10.1145/3691620.3695510.

[6] Startups.com. AI Wrapper. Startup Lexicon, n.d. URL https://www.startups.com/lexicon/ai-wrapper.

[7] OECD. Artificial intelligence and competitive dynamics in downstream markets. Oecd competition law and policy working papers, OECD Publishing, 2025. URL https://www.oecd.org/en/publications/artificial-intelligence-and-competitive-dynamics-in-downstream-markets_ccf0624a-en.html.

[8] Pierre Azoulay, Joshua L. Krieger, and Abhishek Nagaraj. Old moats for new models: Openness, control, and competition in generative AI. Working Paper 32474, National Bureau of Economic Research, 2024. URL https://www.nber.org/papers/w32474. Revised July 2024.

[9] Fernando van der Vlist, Anne Helmond, and Fabian Ferrari. Big AI: Cloud infrastructure dependence and the industrialisation of artificial intelligence. *Big Data & Society*, 11(1), 2024. doi: 10.1177/20539517241232630.

[10] Dieuwertje Luitse. Platform power in AI: The evolution of cloud infrastructures in the political economy of artificial intelligence. *Internet Policy Review*, 2024. doi: 10.14763/2024.2.1768. URL https://policyreview.info/articles/analysis/platform-power-ai-evolution-cloud-infrastructures.

[11] Gabriel Costa Silva, Louis M. Rose, and Radu Calinescu. A systematic review of cloud lock-in solutions. In *2013 IEEE 5th International Conference on Cloud Computing Technology and Science*, volume 2, pages 363–368. IEEE, 2013. doi: 10.1109/CloudCom.2013.130.

[12] Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao, Qi Cai, and Ziyuan Ling. On-device language models: A comprehensive review. arXiv preprint arXiv:2409.00088, 2024. URL https://arxiv.org/abs/2409.00088.

[13] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. MobileLLM: Optimizing sub-billion parameter language models for on-device use cases. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32431–32454. PMLR, 2024. URL https://proceedings.mlr.press/v235/liu24ce.html.

[14] Guanqiao Qu, Qiyuan Chen, Wei Wei, Zheng Lin, Xianhao Chen, and Kaibin Huang. Mobile edge intelligence for large language models: A contemporary survey. *IEEE Communications Surveys & Tutorials*, 27(6):3820–3860, 2025. doi: 10.1109/COMST.2025.3527641.

[15] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 2019. doi: 10.1145/3359591.3359737. URL https://martin.kleppmann.com/2019/10/23/local-first-at-onward.html.

[16] `llama.cpp` contributors. `llama.cpp`. Open-source software, n.d. URL https://github.com/ggerganov/llama.cpp.

[17] llama.rn contributors. llama.rn: React native binding of `llama.cpp`. Open-source software, n.d. URL https://github.com/mybigday/llama.rn/tree/v0.12.9. Version 0.12.9; published package and source inspected for the integration comparison.

[18] Nitro Modules contributors. Nitro modules. Software documentation, n.d. URL https://nitro.margelo.com/.

[19] Irene Solaiman. The gradient of generative AI release: Methods and considerations. In *2023 ACM Conference on Fairness, Accountability, and Transparency*, pages 111–122, 2023. doi: 10.1145/3593013.3593981. URL https://arxiv.org/abs/2302.04844.

[20] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, et al. On the opportunities and risks of foundation models. arXiv preprint arXiv:2108.07258, 2021. URL https://arxiv.org/abs/2108.07258.

[21] Xubin Wang, Zhiqing Tang, Jianxiong Guo, Tianhui Meng, Chenhao Wang, Tian Wang, and Weijia Jia. Empowering edge intelligence: A comprehensive survey on on-device AI models. *ACM Computing Surveys*, 57(9), 2025. doi: 10.1145/3724420. URL https://arxiv.org/abs/2503.06027.

[22] Daliang Xu, Wangsong Yin, Hao Zhang, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. EdgeLLM: Fast on-device LLM inference with speculative decoding. *IEEE Transactions on Mobile Computing*, 24 (4), 2025. doi: 10.1109/TMC.2024.3513457. URL https://yinwangsong.github.io/files/tmc24xu.pdf.

[23] Hugging Face. Text generation models sorted by downloads, n.d.. URL https://huggingface.co/models?pipeline_tag=text-generation&sort=downloads. November 2026 listing; the evaluation data includes a timestamped top-20 API snapshot.

[24] Qwen Team. Qwen3.5-2B model card, n.d. URL https://huggingface.co/Qwen/Qwen3.5-2B.

[25] Google DeepMind. Gemma 4 E2B-it model card, n.d. URL https://huggingface.co/google/gemma-4-E2B-it.

[26] Hugging Face. SmolVLM2-2.2B-Instruct model card, n.d.. URL https://huggingface.co/HuggingFaceTB/SmolVLM2-2.2B-Instruct.

[27] NVIDIA. Cosmos-Reason2-2B model repository, n.d. URL https://huggingface.co/nvidia/Cosmos-Reason2-2B.

[28] OpenBMB. MiniCPM5-2B model card, n.d. URL https://huggingface.co/openbmb/MiniCPM5-2B.

[29] IBM Granite Team. Granite-3.3-2B-Instruct model card, 2025. URL https://huggingface.co/ibm-granite/granite-3.3-2b-instruct.

[30] Tolulope Oshinowo, Sohyeon Hwang, Amy X. Zhang, and Andrés Monroy-Hernández. Seeing the politics of decentralized social media protocols. *Proceedings of the ACM on Human-Computer Interaction*, 10(6):1–27, 2026. doi: 10.1145/3816995. URL https://doi.org/10.1145/3816995. Article CSCW147.

[31] Tolulope Olajubu Oshinowo. Pantry: Reconceptualizing social media feed curation and its implications for user agency. Thesis, Princeton University, 2026. URL https://www.proquest.com/openview/27153640c9a147108271fc5e6e6efe6d/1?pq-origsite=gscholar&cbl=18750&diss=y. ProQuest Dissertations & Theses, publication number 32397157.

[32] Omar El Malki, Marianne Aubin Le Quéré, Andrés Monroy-Hernández, and Manoel Horta Ribeiro. Bonsai: Intentional and personalized social media feeds. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, pages 1–24, 2026. doi: 10.1145/3772318.3791855. URL https://doi.org/10.1145/3772318.3791855. Article 1395.

[33] Jonghyun Chung and Sanket Badhe. Beyond locality: Feature-boundary auditing for OS-integrated AI. arXiv preprint arXiv:2606.10173v2, 2026. URL https://arxiv.org/abs/2606.10173v2. Version 2, October 12, 2026.

[34] Expo. Add custom native code, n.d. URL https://docs.expo.dev/workflow/customizing/.